



Université de Tunis El Manar

École Nationale d'Ingénieurs de Tunis

Département Technologies de l'Information et de la Communication

Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Diplôme National d'Ingénieur en Génie Informatique

**Conception et déploiement d'un pare-feu applicatif
Web intelligent intégrant l'apprentissage
automatique pour la protection des applications
d'entreprise**

Réalisé par :

Baha Eddine **Belhaj MOULDI**

Soutenu devant le jury composé de :

Mme Mariem ZAOUALI	Présidente
Mme Imène ELLOUMI	Rapporteuse (ENIT)
Mme Aida Ben Chehida Douss	Encadrante académique (ENIT)
Mme Rihem OUESLATI	Encadrante professionnelle (Beehive Entreprises)
Mr Maledh MARRAKCHI	Expert industriel invité

Entreprise d'accueil :

Beehive Entreprises



Année universitaire : 2025–2026

*À nos chers parents,
pour leurs sacrifices, leur amour
et leur soutien sans faille.*

*À nos frères et sœurs,
dont la présence et les mots simples
nous ont souvent aidé à poursuivre.*

*À celles et ceux qui ont partagé nos années d'études,
les longues journées de travail comme les instants plus légers,
et qui ont rendu ce parcours profondément humain.*

*À tous ceux qui, de près ou de loin,
ont contribué à l'aboutissement
de ce travail.*

Nous dédions ce modeste travail.

Remerciements

Ce mémoire clôt plusieurs mois consacrés à la conception, aux essais et à la mise en production de BeeWAF. Ce travail n'aurait pas pris la même forme sans les personnes qui nous ont conseillé, ouvert leur environnement de travail et aidé à prendre du recul lorsque les choix techniques devenaient difficiles.

Nous remercions tout particulièrement **Mme Aida Ben Chehida Douss**, encadrante académique à l'**ENIT**. Ses remarques nous ont permis de mieux structurer la démarche, de préciser nos arguments et de présenter les résultats avec davantage de rigueur. Nous lui sommes reconnaissants pour la qualité de son suivi et pour l'attention portée aux différentes versions du rapport.

Nous exprimons également notre profonde reconnaissance à **Mme Rihem OUESLATI**, encadrante professionnelle chez **Beehive Entreprises**. Elle nous a donné accès à un contexte technique réel, fait confiance lors des déploiements et accompagné dans les décisions qui engageaient la production. Sa disponibilité, ses retours directs et sa connaissance de l'infrastructure ont été essentiels à l'avancement du projet.

Nous n'oublions pas l'équipe technique de **Beehive Entreprises**, en particulier les ingénieurs DevOps et les développeurs des projets *IDTS* et *Kairos*. Leur écoute, leur partage de savoir-faire et une atmosphère de travail réellement collaborative ont rendu notre intégration aussi simple qu'agréable.

Nous remercions enfin les membres du **jury** pour le temps consacré à l'examen de ce mémoire : **Mme Mariem ZAOUALI**, qui nous fait l'honneur de présider ce jury, **Mme Imène ELLOUMI**, qui a accepté d'en être la rapporteure, ainsi que **Mr Maledh MARRAKCHI**, qui nous fait l'honneur d'y prendre part en qualité d'expert industriel invité. Leurs questions et leur regard extérieur permettront de prolonger la réflexion engagée durant le stage.

Nous tenons enfin à saluer l'ensemble des enseignants et du personnel administratif du département **TIC** de l'**ENIT** : c'est sur les fondations posées au fil de ces années d'études que repose aujourd'hui ce projet.

À notre famille et à nos amis, pour leur patience et leurs encouragements durant cette période exigeante, va notre reconnaissance la plus profonde.

Résumé

Aujourd'hui, l'essentiel des services des organisations repose sur des applications accessibles en ligne, ce qui en fait une cible de premier choix pour les attaquants. À mesure que les offensives se raffinent et que les volumes de trafic à inspecter augmentent, une défense qui ne s'appuie que sur des signatures statiques se révèle vite insuffisante.

Le projet mené chez **Beehive Entreprises** répond à ce besoin par la réalisation de **BeeWAF**, un pare-feu applicatif Web conçu pour l'infrastructure de l'entreprise et déployé en production dans sa version **V-8.54**. Son pipeline Kubernetes regroupe **vingt-sept contrôles successifs**. La détection associe 107 signatures issues de 9 824 motifs regex, un ensemble *Random Forest/Gradient Boosting* exploitant 38 caractéristiques et un mécanisme de normalisation capable de défaire 18 couches d'encodage. BeeWAF complète ce socle par le renseignement sur les menaces, l'analyse heuristique des attaques inconnues, 37 correctifs virtuels de CVE et une interface dédiée au traitement des faux positifs. Le suivi de sept référentiels de conformité est intégré à la solution. En exploitation, les métriques sont collectées par Prometheus et visualisées dans Grafana. La chaîne ELK a été préparée, mais son déploiement a été reporté en raison des ressources disponibles sur le cluster.

L'évaluation repose sur 153 scénarios offensifs représentatifs de l'OWASP Top 10, rejoués avec *Burp Suite Professional*, notamment sous plusieurs formes encodées. Sur les 12 213 requêtes du jeu de test CSIC 2010, l'ensemble ML obtient 92,3 % d'*accuracy*, 98,2 % de *precision* et 89,8 % de score F1. Les observations en production montrent par ailleurs que le traitement propre à BeeWAF ajoute moins de 20 ms à la latence médiane.

Mots-clés : Pare-feu applicatif Web, WAF, OWASP, apprentissage automatique, sécurité applicative, Kubernetes, DevSecOps, détection d'intrusion, faux positifs.

Abstract

Most of an organisation’s services now live on the Web, which makes it a prime target for attackers. As offensives become more refined and the volume of traffic to inspect keeps growing, a defence resting on static signatures alone quickly falls short.

The work conducted at **Beehive Entreprises** addresses this need through **BeeWAF**, a Web Application Firewall tailored to the company’s infrastructure and released to production as version V-8.54. Its Kubernetes pipeline performs **twenty-seven consecutive checks**. Detection combines 107 signatures assembled from 9,824 regex patterns, a *Random Forest/Gradient Boosting* ensemble using 38 features, and a normalization mechanism that unwraps up to 18 encoding layers. Threat intelligence, heuristic unknown-attack detection, virtual patches for 37 CVEs, and a dedicated false-positive management interface complete the design. The system also maps its controls to seven compliance frameworks. Operational monitoring uses Prometheus and Grafana; the ELK pipeline was configured but not deployed because of the cluster’s resource constraints.

Evaluation used 153 offensive scenarios representative of the OWASP Top 10 and replayed with *Burp Suite Professional*, including several encoded variants. On the 12,213-request CSIC 2010 test split, the ML ensemble achieved 92.3% accuracy, 98.2% precision, and an F1 score of 89.8%. Production measurements also showed that BeeWAF itself adds less than 20 ms to median request latency.

Keywords: Web Application Firewall, WAF, OWASP, machine learning, application security, Kubernetes, DevSecOps, intrusion detection, false positives.

Table des matières

Table des figures	ix
Liste des tableaux	xi
Liste des Acronymes	xiii
Introduction Générale	1
1 Présentation générale du projet	3
Introduction	3
1.1 Présentation de l'entreprise d'accueil : Beehive Entreprises	3
1.1.1 Présentation générale de Beehive Entreprises	3
1.1.2 Domaines d'expertise de Beehive Entreprises	4
1.1.3 Organisation interne et environnement de travail de Beehive Entreprises	4
1.2 Présentation du cadre du projet	5
1.2.1 Contexte général du projet	5
1.2.2 Problématique et enjeux de la sécurité applicative	6
1.3 Étude de l'existant chez Beehive Entreprises	7
1.3.1 Architecture applicative de l'entreprise	7
1.3.2 Cluster Kubernetes et flux de trafic	8
1.3.3 Critique de l'existant	9
1.4 Spécification du projet	9
1.4.1 Objectif général du projet	9
1.4.2 Exigences fonctionnelles	9
1.4.3 Exigences non fonctionnelles	10
1.5 Méthodologie de gestion du projet	11
1.5.1 Démarche agile adaptée	11
1.5.2 Outillage utilisé	12
1.5.3 Planification du projet	12
Conclusion	13

2	Étude des concepts de base théoriques et de l'existant	14
	Introduction	14
2.1	Notions fondamentales sur les WAF	14
2.1.1	Définition et rôle d'un WAF	14
2.1.2	Modes de déploiement d'un WAF	15
2.1.3	Stratégies de détection d'un WAF	15
2.2	Panorama des solutions WAF existantes	16
2.2.1	Solutions WAF commerciales	16
2.2.2	Solutions WAF <i>cloud</i>	19
2.2.3	Solutions WAF open-source	21
2.2.4	Étude comparative des solutions présentées	22
2.3	Apprentissage automatique pour la sécurité applicative	23
2.3.1	Apport du ML pour la sécurité applicative	23
2.3.2	Modèles supervisés couramment utilisés	23
2.3.3	Jeux de données de référence	24
2.3.4	Indicateurs d'évaluation	24
2.4	Architecture cloud-native et briques d'infrastructure	25
2.4.1	Conteneurisation avec Docker	25
2.4.2	Orchestration avec Kubernetes	26
2.4.3	Approche GitOps avec ArgoCD	26
2.4.4	Observabilité avec Prometheus et Grafana	27
2.5	Récapitulatif des choix techniques retenus	27
	Conclusion	28
3	Conception et architecture de BeeWAF	29
	Introduction	29
3.1	Architecture globale de BeeWAF	29
3.2	Modélisation UML	31
3.2.1	Diagramme de cas d'utilisation	31
3.2.2	Diagramme de classes	32
3.2.3	Diagramme de séquence — inspection d'une requête	33
3.2.4	Diagramme de séquence — classification d'un faux positif	34
3.3	Pipeline de traitement des requêtes	34
3.4	Modules de protection	36
3.4.1	Validation de protocole	36
3.4.2	Géo-blocage et réputation IP	37
3.4.3	Détection de bots (deux étages)	37
3.4.4	Protection DDoS multi-couches	38
3.4.5	Renseignement sur les menaces	38

3.4.6	Détecteur d'évasion (<i>Evasion Detector</i>)	39
3.4.7	Moteur de règles regex	40
3.4.8	Protection de session et des cookies	41
3.4.9	Sécurité et découverte d'API	42
3.4.10	<i>Virtual patching</i> multi-CVE	42
3.4.11	Inspection WebSocket et analyse de <i>payload</i>	42
3.4.12	Détecteur <i>zero-day</i> heuristique	43
3.4.13	Moteur de corrélation et <i>Cyber Kill Chain</i>	43
3.4.14	Apprentissage adaptatif (<i>positive security</i>)	44
3.4.15	DLP et masquage de réponses	44
3.4.16	Allowlist automatique et registre de routes	44
3.4.17	Moteur de conformité multi-référentiels	44
3.4.18	Performance et haute disponibilité	45
3.5	Moteur d'apprentissage automatique	45
3.5.1	Architecture de l'ensemble	45
3.5.2	Extraction de 38 caractéristiques	46
3.5.3	Jeux de données d'entraînement	47
3.5.4	Cycle de vie et auto-réentraînement	48
3.5.5	Entraînement asynchrone et sauvegarde atomique	49
3.6	Tableau de bord de gestion des faux positifs (FP Dashboard)	49
3.6.1	Architecture du tableau de bord	49
3.6.2	Onglets et fonctionnalités	49
3.7	Observabilité	51
3.7.1	Métriques Prometheus	51
3.7.2	Tableaux de bord Grafana et pile ELK	52
	Conclusion	52
4	Réalisation et résultats obtenus	53
	Introduction	53
4.1	Environnement de développement et de production	53
4.1.1	Environnement matériel	53
4.1.2	Environnement logiciel	54
4.2	Mise en œuvre du pipeline CI/CD	54
4.2.1	Pipeline Jenkins	55
4.2.2	Conteneurisation et entraînement ML au build	57
4.2.3	Déploiement GitOps avec ArgoCD	57
4.3	Déploiement et opération sur Kubernetes	58
4.3.1	Manifestes Kubernetes	58
4.3.2	Persistance des données	59

4.3.3	Collecte de journaux avec ELK	59
4.4	Tableau de bord Grafana	60
4.5	Tests de sécurité	60
4.5.1	Méthodologie de test	61
4.5.2	Couverture OWASP Top 10	61
4.5.3	Tests complémentaires avec les outils Kali Linux	62
4.5.4	Tests d'évasion par encodage	66
4.5.5	Tests de performance du moteur ML	66
4.5.6	Performances opérationnelles en production	67
4.6	Bilan du FP Dashboard	69
4.7	Discussion des résultats	71
4.7.1	Points forts	71
4.7.2	Limites identifiées	72
4.7.3	Évolutions apportées durant le stage	72
	Conclusion	73
	Conclusion Générale	74
	Bibliographie	77